Session 5 · Deck 5.B1

Projection: The Residual

Zhigang Feng

Thursday, October 22, 2026

© Zhigang Feng • Redistribution prohibited without permission

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 5.B1 ▼

S1 ✓	S2 ✓	S3 ✓	S4 ✓	S5	S6	S7	S8	S9	S10
Sep 24	Oct 1	Oct 8	Oct 15	Oct 22	Oct 29	Nov 5	Nov 12	Nov 19	Dec 3
Intro	HA · OLG	Num. DP	Numerics	Numerics	HA classic	ML	RL	Text	Agents
Growth	Policy	Python	I	II	The wall	Deep nets	HA + DL	LLMs	Projects
trunk	branches	the machine · classical methods			the wall	new tools		new data	colleagues

Build it on paper → solve it on the machine → push it to the frontier with AI

Block B — projection: **5.B1** the residual · 5.B2 choosing a basis · 5.B3 many dimensions and finite elements · 5.B4 coefficients and the worked example.

© Zhigang Feng
Redistribution prohibited without permission

Roadmap: Three Parts

Block A solved the model near one point. Block B solves it on the whole domain, by writing the unknown function as a weighted sum of known shapes and making the equation's error small.

Part	Topic	The question it answers
1	The idea	What is the unknown now, and what does "solved" mean?
2	The algorithm and the residual	What do we actually compute, and what does its error look like?
3	Against perturbation; the two choices ahead	When is this the right road, and what is left to decide?

© Zhigang Feng Redistribution prohibited without permission

By the end: "solve the model" will mean "choose a basis, then choose coefficients that make the residual small", and you will know the two decisions the rest of the block is about.

Three methods, three answers to "what is the unknown?" Projection's answer is a short list of coefficients.

Block A bought speed by giving up **global validity**: the solution is exact at the steady state and drifts as we move away.

Block B makes the opposite trade. Keep the solution valid on the **whole domain** — but stop storing one number per grid point, as Sessions 3–4 did.

The unknown becomes a short vector of **coefficients**, and "solved" means

the residual is small.

Method	The unknown is
VFI (S3–S4)	N values on a grid
perturbation	derivatives at one point
projection	**coefficients on a basis**

Idea

A table of values is a very long list of unknowns. A formula with a few free numbers in it is a very short one. Projection chooses the formula first and solves for the numbers.

© Zhigang Feng · Redistribution prohibited without permission

The same growth model as Block A, now with a labour choice: two unknown functions instead of one.

$$\text{(Euler)} \quad \frac{1}{c_t} = \beta \mathbb{E}_t \left[\frac{1}{c_{t+1}} \left(1 + \alpha k_{t+1}^{\alpha-1} (e^{z_{t+1}} l_{t+1})^{1-\alpha} - \delta \right) \right]$$

$$\text{(Labour)} \quad \psi \frac{c_t}{1 - l_t} = (1-\alpha) k_t^{\alpha} (e^{z_t} l_t)^{1-\alpha} \frac{1}{l_t}$$

$$\text{(Resource)} \quad c_t + k_{t+1} = k_t^{\alpha} (e^{z_t} l_t)^{1-\alpha} + (1-\delta) k_t$$

$$\text{(Shock)} \quad z_t = \rho z_{t-1} + \varepsilon_t$$

The objective is the one 5.A1 set: find d with $H(d) = 0$, where now

$$k_{t+1} = d_1(k_t, z_t), \qquad l_t = d_2(k_t, z_t).$$

Read the equations

Euler: give up one unit today, get $1 + r_{t+1}$ tomorrow. Labour: the wage buys leisure at the rate $\psi c/(1-l)$. Resource: output is consumed or saved.

Read the indices

Everything inside the expectation carries $t+1$: the shock and labour inside the marginal product too. A wrong index still runs; it solves a different model.

© Zhigang Feng Redistribution prohibited without permission

The Core Idea: A Few Numbers Stand In for a Function

Pick a family with finitely many free coefficients; the equation's error as a function of the state is the residual.

Restrict the unknown function to a finite family:

$$d^n(x;\theta) \;=\; \sum_{i=0}^{n} \underbrace{\theta_i}_{\text{unknown weight}} \underbrace{\psi_i(x)}_{\text{known shape}}$$

$\{\psi_i\}$ is a chosen **basis** (polynomials, splines, tents): known shapes. $\{\theta_i\}$ are the $n+1$ **coefficients** we solve for: how much of each shape.

Substituting into the equilibrium conditions gives the **residual**,

$$R(x;\theta) = H\big(d^n(x;\theta)\big),$$

the amount by which the equations fail at state x when the weights are θ.

Idea

Think of a regression: the ψ_i are regressors, the θ_i coefficients, R the error to make small. Here there is no data; the model's equations play its role.

The basis need not be fixed in advance. Replace $\sum_i \theta_i \psi_i$ by a neural network $\mathcal{N}(x;\theta)$ and nothing else changes — that is Session 7.

© Zhiguang Feng Redistribution prohibited without permission

1 · Why the Name "Projection"

The true solution lives in an infinite-dimensional space of functions. We shrink the problem to $n+1$ numbers, so the equation can only hold in a finite sense.

The true d could be any smooth function. Choosing a basis **projects** the problem onto the functions the basis can produce.

With only $n+1$ coefficients we cannot make $R(x;\theta) = 0$ at *every* x. We impose finitely many conditions instead:

- **Collocation:** $R(x_j;\theta) = 0$ at $n+1$ chosen points.
- **Galerkin:** $\langle R, \psi_i \rangle = 0$ for each i — the residual is orthogonal to every basis function.
- **Least squares:** minimise $\|R\|$ over the domain.

Each is a sense in which the residual has been *projected away*.

The result is **global**: valid over the whole domain, not a neighbourhood — provided the basis can represent the function and the projection is sensibly chosen.

Compare Block A: perturbation imposes conditions on *derivatives at one point*. Projection imposes conditions on *values over a domain*. Same equation $H = 0$, opposite discipline.

© Zhigang Feng · Redistribution prohibited without permission

The Algorithm

Five lines. The first four are bookkeeping; the fifth is a nonlinear system in the coefficients, and Session 4's solvers do it.

1: choose $n+1$ linearly independent basis functions $\psi_i : \Psi \to \mathbb{R}^m$
2: unknowns: $\theta = [\theta_0, \ldots, \theta_n]$
3: approximant: $d^n(\cdot;\theta) = \sum_{i=0}^{n} \theta_i \psi_i(\cdot)$
4: residual: $R(\cdot;\theta) = H\big(d^n(\cdot;\theta)\big)$ ▷ a function of x
5: $\hat{\theta} \leftarrow \arg\min_\theta \ \rho\big[R(\cdot;\theta)\big]$ ▷ ρ: what "small" means

Line 5 is the computation. The metric ρ is a choice: zero at points, zero against the basis, or least squares (5.B4).

Where the work is

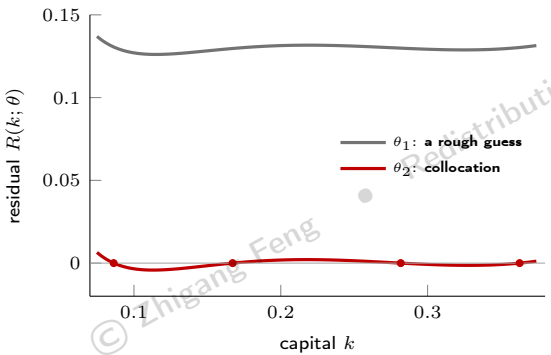
Line 5 is a **nonlinear system** in θ: as many equations as coefficients. Newton or a quasi-Newton method solves it, so 4.A1 and 4.A2 apply directly.

"Linearly independent" means no basis function is a combination of the others — otherwise two different θ would give the same d^n and the system would have no unique answer.

© Zhigang Feng · Redistribution prohibited without permission

The Residual Is a Function — Look at It

Same basis, two coefficient vectors. The basis fixes a family of curves; θ picks one member, and the residual says how wrong that member is at every k.



© Zhigang Feng
Redistribution prohibited without permission

Block A's model (log utility, full depreciation), four Chebyshev coefficients in k.

θ_1 is a rough guess, a policy 15% too high: its residual reaches 0.137. θ_2 is chosen to zero the residual at four points (the dots): its largest residual anywhere is 0.0064.

That is all projection does: search over θ for the member of the family whose residual is smallest, in the sense ρ says. Between the dots the residual is not zero — it is just small.

Two roads to the same unknown: one local and fast, one global and slower. They are partners, not rivals.

	Perturbation (Block A)	**Projection (Block B)**
Approximates	locally, around one point	globally, on a domain
Unknowns are	derivatives at that point	coefficients on a basis
Conditions	derivatives of F vanish	R small: at points, against the basis, or in norm
Speed	very fast; linear algebra	a nonlinear solve; slower
Dimension	tractable in large state spaces (local information only)	the curse of dimensionality returns
Nonlinearity	accuracy degrades away from the point	handled directly; no expansion

© Zhigang Feng Redistribution prohibited without permission

Mixed approaches

Perturbation knows the policy almost exactly at one point; projection knows it well everywhere. Perturbation supplies the **initial guess** for the projection solve — 5.B4's worked example does exactly this.

Every named projection method in the literature is one pair of answers to two independent questions.

Choice 1: which basis?

Global basis — each ψ_i is nonzero over the whole domain (polynomials): *spectral methods*.

Local basis — each ψ_i is nonzero only on a small piece (tents): *finite elements*.

→ 5.B2 (one dimension), 5.B3 (many dimensions; finite elements).

Choice 2: how to project?

What does "the residual is small" mean — zero at chosen points, zero against each basis function, or smallest in norm?

→ 5.B4, where six answers are tried on one model.

Different choices give slightly different methods. The framework is one framework, and it mixes with perturbation.

© Zhigang Feng · Redistribution prohibited without permission

Takeaways

- **Part 1.** Projection replaces the unknown *function* by a weighted sum of known shapes, $d^n = \sum_i \theta_i \psi_i$, and redefines "solved" as **the residual is small**. The result is global: valid on the whole domain, nonlinearity included.
- **Part 2.** The algorithm is five lines; the work is a nonlinear system in θ (Session 4's Newton). The residual is a **function**: zeroing it at four points left it at most 0.0064 in between — small, not zero.
- **Part 3.** Perturbation is local and fast; projection is global and slower, and the curse of dimensionality returns. They mix: perturbation's answer is projection's starting guess. Two choices remain — the basis (5.B2, 5.B3) and the meaning of "small" (5.B4).

© Zhigang Feng · Redistribution prohibited without permission

Bridge: Which Basis?

Everything above was written with an unspecified $\{\psi_i\}$, and the picture quietly used Chebyshev polynomials without saying why.

The choice matters more than it looks. The obvious basis — $1, x, x^2, \ldots$ — fails, for a reason that has nothing to do with how well polynomials can approximate and everything to do with how the equations for θ behave on a computer.

Next (5.B2)

Global bases and what happens at a kink; why monomials are unusable; what "orthogonal" buys; and the Chebyshev polynomials, the default basis of the field.

© Zhigang Feng · Redistribution prohibited without permission